

React Advanced + Next.js Cheatsheet

SPA — Single Page Application

- একটাই HTML file load হয়, বাকি সব JavaScript দিয়ে change হয়
 - Page reload হয় না — শুধু data আসে, UI React আঁকে
 - 💡 "UI change without full page reload"
-

Client-side Routing

- URL change হলে server-এ request যায় না — React Router নির্দিষ্ট component render করে
 - প্রতিটা route = একটা component
 - 💡 URL → Component mapping
-

Nested Routing + Outlet

- Parent route = layout (Navbar, Sidebar)
 - Child route = specific content
 - `<Outlet />` = parent layout-এর ভেতরে child content বসার জায়গা
 - 💡 UI nesting = Route nesting
-

Link vs NavLink

	Link	NavLink
কাজ	Navigate করে	Navigate + active detect করে
কোথায়	যেকোনো জায়গায়	Navbar, sidebar

Data Mode — Core Idea

React Router Data Mode-এ route আর শুধু UI না — **data + UI** একসাথে **define** হয়

- `createBrowserRouter([])` → route object array
 - `RouterProvider` → app-এ router inject করে
 - পুরনো `<BrowserRouter>` + `<Routes>` বাদ দিয়ে এই pattern ব্যবহার করো
-

Loader + useLoaderData

- Component render হওয়ার আগেই data fetch হয়
 - `loader` function route-এর সাথে define হয়
 - Component-এ `useLoaderData()` দিয়ে data পাওয়া যায়
 - `useEffect` fetch-এর দরকার নেই
 - 💡 Route loads data → Component just displays it
-

Dynamic Route + Params

- Static: `/about` — সবসময় একই
 - Dynamic: `/product/:id` — same component, ভিন্ন data
 - `loader({ params })` → URL-এর id দিয়ে data fetch করা যায়
 - 💡 URL = data source
-

useNavigate vs useNavigation

Hook	কাজ
<code>useNavigate()</code>	Programmatically অন্য route-এ যাওয়া (login redirect)
<code>useNavigation()</code>	Router-এর loading state জানা (spinner দেখানো)

useLocation

- বর্তমান URL-এর সব তথ্য দেয়
 - Login-এর পর আগের page-এ ফিরে যেতে কাজে লাগে
 - 💡 `location.state` দিয়ে "কোথা থেকে এলাম" মনে রাখা যায়
-

Error + Loading + 404

- `errorElement` → loader throw করলে fallback UI
 - `HydrateFallback` → data load হওয়ার সময় skeleton দেখায়
 - `path="*" → সব undefined route ধরে 404 দেখায়`
-
-

Form Handling

- `onSubmit + e.preventDefault()` → browser reload বন্ধ করে data নেওয়া
 - **Controlled** → input value state দিয়ে control — validation সহজ
 - **Uncontrolled** → DOM নিজে value রাখে, `ref` দিয়ে পড়া হয়
 - 💡 Controlled = state driven / Uncontrolled = DOM driven
-

useRef

- DOM element সরাসরি access করতে
 - Value store করা যায় কিন্তু re-render হয় না
 - 💡 `state` → re-render ঘটায় / `ref` → ঘটায় না
-

Custom Hook

- Reusable logic আলাদা function-এ রাখা
 - নাম সবসময় `use` দিয়ে শুরু
 - একই logic অনেক component-এ ব্যবহার করা যায়
 - Example: `useFetch`, `useForm`, `useAuth`
 - 💡 Custom Hook = Logic extract করার tool
-

Prop Drilling সমস্যা

- অনেক level নিচে data পাঠাতে হলে মাঝের সব component-এ props দিতে হয়
 - Code messy হয়, maintain করা কঠিন
 - সমাধান → **Context API**
-

Context API

- Global data (user, theme, cart) সব component-এ share করা যায়
 - তিনটি ধাপ: `createContext` → `Provider` → `useContext`
 - 💡 `Provider` দিয়ে wrap করে, যেকোনো জায়গা থেকে use করে
-

Context API vs Custom Hook

	Context API	Custom Hook
মূল কাজ	Global data share	Logic reuse
Scope	App-wide	Component level
Example	auth, theme	useFetch, useForm

💡 **Context = data / Hook = logic** — দুটো একসাথেও ব্যবহার হয়

Next.js কেন?

- React-এর উপর built framework
 - SSR, SEO, performance, file-based routing — সব built-in
 - 💡 `React = library / Next.js = full framework`
-

File-based Routing

- Folder structure-ই route — আলাদা router config নেই
 - `app/about/page.tsx` → `/about`
 - `app/blog/[id]/page.tsx` → `/blog/123`
 - `app/docs/[...slug]/page.tsx` → `/docs/a/b/c`
-

Special Files

File	কাজ
<code>page.tsx</code>	Route-এর main UI
<code>layout.tsx</code>	Shared wrapper — Navbar, Footer
<code>loading.tsx</code>	Data আসার আগে skeleton/spinner
<code>error.tsx</code>	Error হলে fallback UI
<code>not-found.tsx</code>	404 page

Rendering Strategies

পদ্ধতি	কোথায় render	কখন	SEO
CSR	Browser	Runtime	❌ দুর্বল
SSR	Server	প্রতি request	✅ ভালো

Hydration

- Server থেকে static HTML আসে → browser দ্রুত দেখায়
 - তারপর JavaScript attach হয় → app interactive হয়
 - এই process-ই **Hydration**
 - 💡 `Static shell` → `JS attach` → `Interactive`
-

Server vs Client Component

- **Server Component** (default) → data fetch করে, bundle size কম, interactive নয়
 - **Client Component** ("use client") → `useState`, `useEffect`, event handler ব্যবহার করা যায়
 - Server Component-এর ভেতরে Client Component রাখা যায়
 - 💡 যতটা সম্ভব Server Component রাখো — performance ভালো থাকে
-

Search & Filtering

পদ্ধতি	কীভাবে	সীমাবদ্ধতা
Client-side	Local array filter	শুধু loaded data
Server-side	URL param → server fetch	সব data handle করতে পারে

- URL-driven search: `?search=react&category=book`
 - **Shareable, bookmarkable, SEO friendly**
 - Server Component-এ `searchParams` prop থেকে query পড়া হয়
-

Master Concept Map

SPA → React Router (Data Mode)

- **createBrowserRouter** → route array
- **loader** → data before render
- **Outlet** → nested UI
- **useNavigate** / **useNavigation** / **useLocation**
- **errorElement** / **HydrateFallback** / **path="*"**

React Patterns

- **Form** → **Controlled** / **Uncontrolled**
- **useRef** → DOM access, no re-render
- **Custom Hook** → logic reuse
- **Context API** → global data, no prop drilling

Next.js

- **File-based Routing** → folder = route
- **Rendering** → CSR / SSR / SSG / ISR
- **Server Component** (default) vs **Client Component**
- **Special files** → loading, error, not-found, layout
- **URL-driven search** → **searchParams** → server fetch